

ECE103L

Visualizing signal in Matlab; Assignment 3

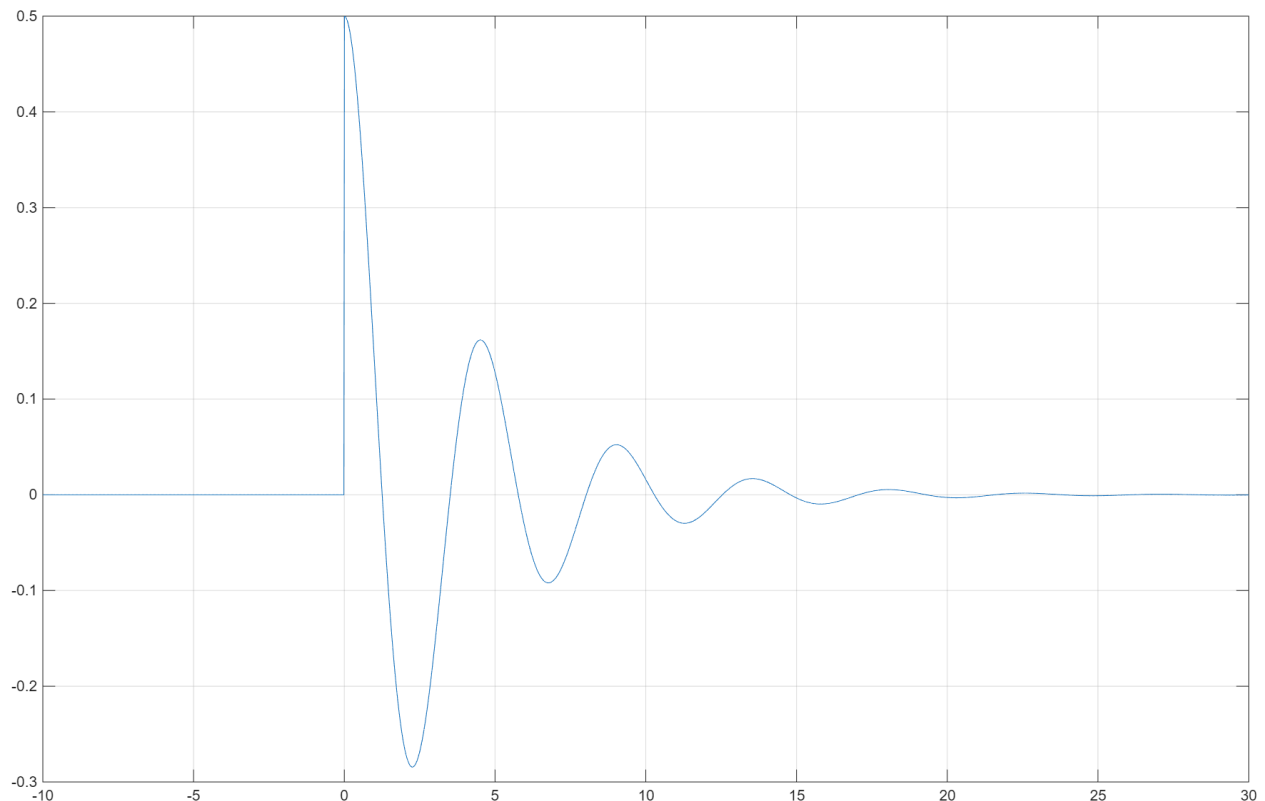
Name: Alec Levy

Date: 4/21/26

1. Following RLC circuit is described by the differential equation (1). Use Matlab built-in differential equation solver `dsolve()` to derive the impulse response $h(t)$ for this circuit when $R=2\Omega$, $C=1F$, $L=0.5 H$. Plot the impulse response $h(t)$ from a range $-10 \leq t \leq 30$.

- a. Summary/Discussion: The impulse response was derived using `dsolve` with initial conditions $h(0) = 0.5$ and $h'(0) = 0$. The result is a damped sinusoid that decays toward zero. The oscillatory behavior comes from the inductor and capacitor exchanging energy, with the resistor dissipating it.

- b. Results:



```

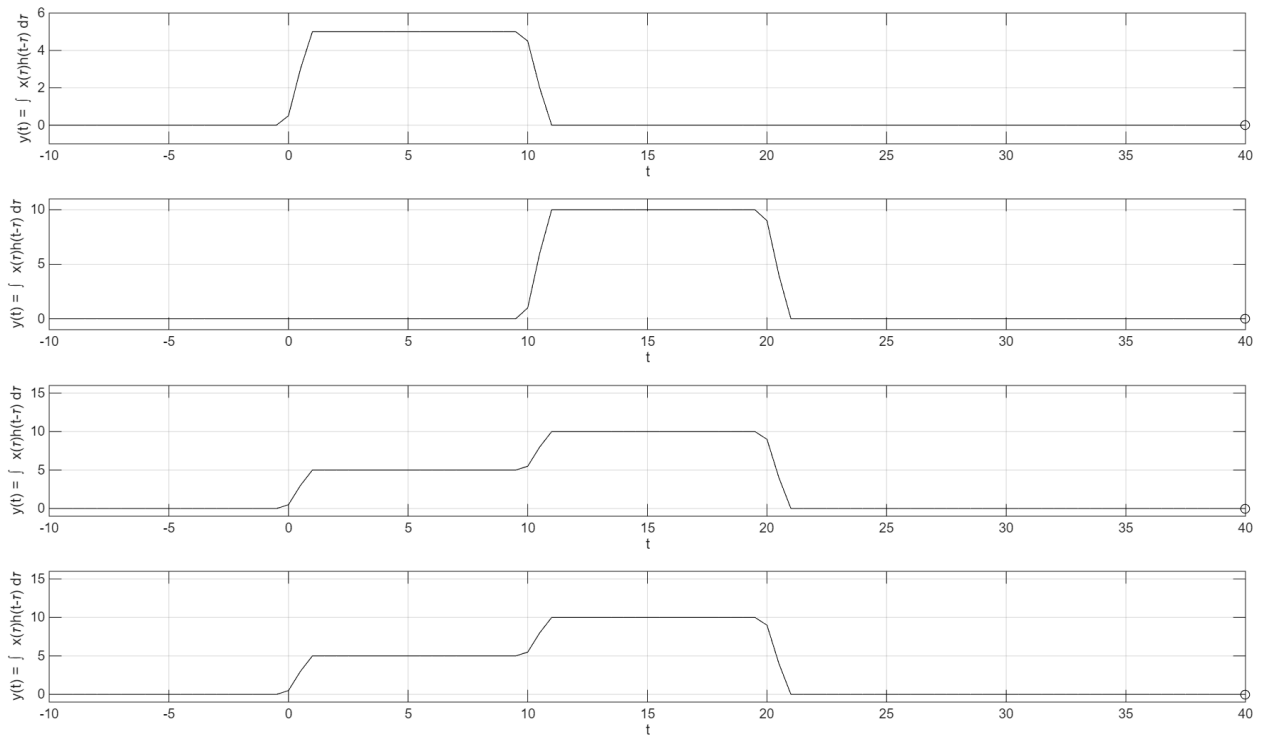
c. %q1
R = 2;
C = 1;
L = 0.5;
syms h(t);
eqn = R*C*diff(h,t,2) + diff(h,t) + (R/L)*h == 0;
Dh = diff(h, t);
cond1 = h(0) == 0.5;
cond2 = Dh(0) == 0;
hsol = dsolve(eqn, cond1, cond2);
hfunc = matlabFunction(hsol);
tvals = linspace(-10, 30, 2000);
hvals = zeros(size(tvals));
idx = tvals >= 0;
hvals(idx) = hfunc(tvals(idx));
plot(tvals, hvals)
grid on

```

2. Using the example Matlab file `simplified_convolution_runtime.m`, plot the output signals in three separate figure windows: (a) $y_1(t) = x_1(t) * h(t)$ (b) $y_2(t) = x_2(t) * h(t)$ (c) $y_{\text{linear_comb}} = x_{\text{linear_comb}}(t) * h(t)$. Use the ranges of ' τ ' and ' t ' as $-10 \leq \tau \leq 40$ and $-10 \leq t \leq 40$. Also plot $y_3(t) = y_1(t) + y_2(t)$ and comment on similarity of $y_3(t)$ and $y_{\text{linear_comb}}(t)$

a. Summary/Discussion: The convolution was computed using `trapz` over a shared τ axis, and their linear combination each convolved against the rectangular impulse response $h(t)$. The outputs ramp up, hold constant while x is active, and then ramp back down. Additionally, the linearity property stands verified because $y_3 = y_1 + y_2$ matches y_4 directly.

b. Results:



c. Code:

```
%q2
close all
clear all
clc
clf % clear figure
% signal: x, impulse response: h
x1 = @(time) 5 * (time >= 0 & time < 10);
x2 = @(time) 2*x1(time-10);
xlinear_comb = @(time) x1(time) + 2*x1(time-10);
h = @(time) (time>=0 & time<1);

dtau = 0.1;
tau = -10:dtau:40;

dT=0.5;
t = -10:dT:40; % time variable for the time delay and output signal
y1 = NaN(1, length (t)); % Pre-allocate memory with NaN (Not a Number)
y2 = NaN(1, length (t));
y3 = NaN(1, length (t));
y4 = NaN(1, length (t));
for ii=1:length(t)
    y1(ii) =trapz(tau, h(t(ii)-tau).*x1(tau));
    y2(ii) =trapz(tau, h(t(ii)-tau).*x2(tau));
    y3(ii) =y1(ii) + y2(ii);
```

```

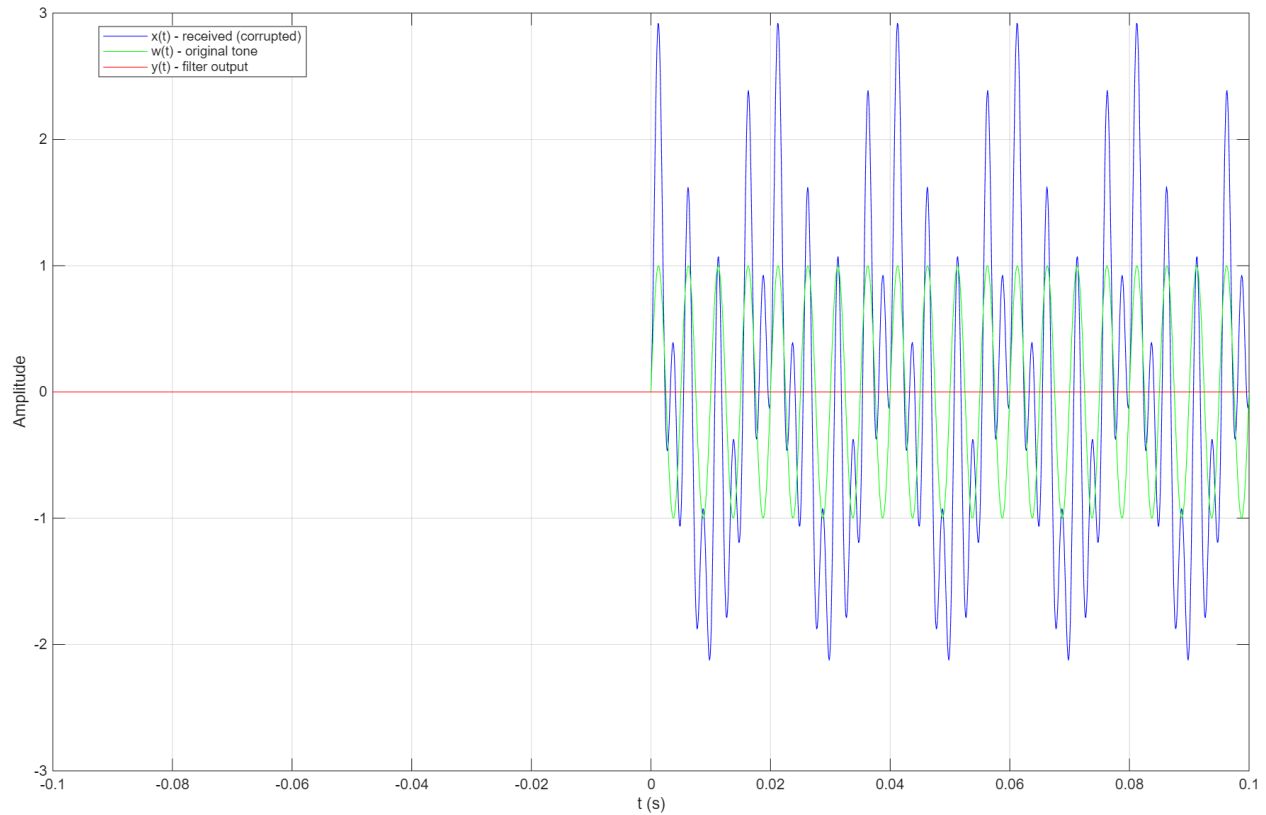
y4(ii) =trapz(tau, h(t(ii)-tau).*xlinear_comb(tau));
end
subplot (411)
plot (t, y1, 'k', t (ii), y1(ii), 'ok');
xlabel ('t'); ylabel ('y(t) = \int x(\tau)h(t-\tau) d\tau');
axis ([tau(1) tau(end) -1.0 6.0]); grid;
drawnow;

subplot (412)
plot (t, y2, 'k', t (ii), y2(ii), 'ok');
xlabel ('t'); ylabel ('y(t) = \int x(\tau)h(t-\tau) d\tau');
axis ([tau(1) tau(end) -1.0 11.0]); grid;
drawnow;
subplot (413)
plot (t, y3, 'k', t (ii), y3(ii), 'ok');
xlabel ('t'); ylabel ('y(t) = \int x(\tau)h(t-\tau) d\tau');
axis ([tau(1) tau(end) -1.0 16.0]); grid;
drawnow;
subplot (414)
plot (t, y4, 'k', t (ii), y4(ii), 'ok');
xlabel ('t'); ylabel ('y(t) = \int x(\tau)h(t-\tau) d\tau');
axis ([tau(1) tau(end) -1.0 16.0]); grid;
drawnow;

```

3. A single-tone signal $w(t) = \sin(400\pi t)$ is transmitted to an audio amplifier and speaker to produce a high-temperature warning for a silicon crystal-growing factory. A filter having impulse response $h(t) = 400e^{-200t} \cos(400\pi t)u(t)$ has been designed to reduce additive interference in the received signal. Using Matlab in-built convolution function: `conv()`, find the filter output signal $y(t)$, when the received signal is $x(t) = [\cos(100\pi t) + \sin(400\pi t) - \cos(800\pi t)]u(t)$ (signal $w(t)$ was corrupted by interference and resulted in an input signal $x(t)$). Plot the output signal, the input signal, and $w(t)$ for the range of $-0.1 \leq t \leq 0.1$. Comment on the effect of the filter on the signal. While solving this problem, pay attention to the time resolution/step (dT) you need to use

- a. Summary/Discussion: The received signal contains three separate frequency components. These were suppressed down to a single frequency that allowed for only one noise to be made. This makes sense and tracks with the graph.
- b. Results:

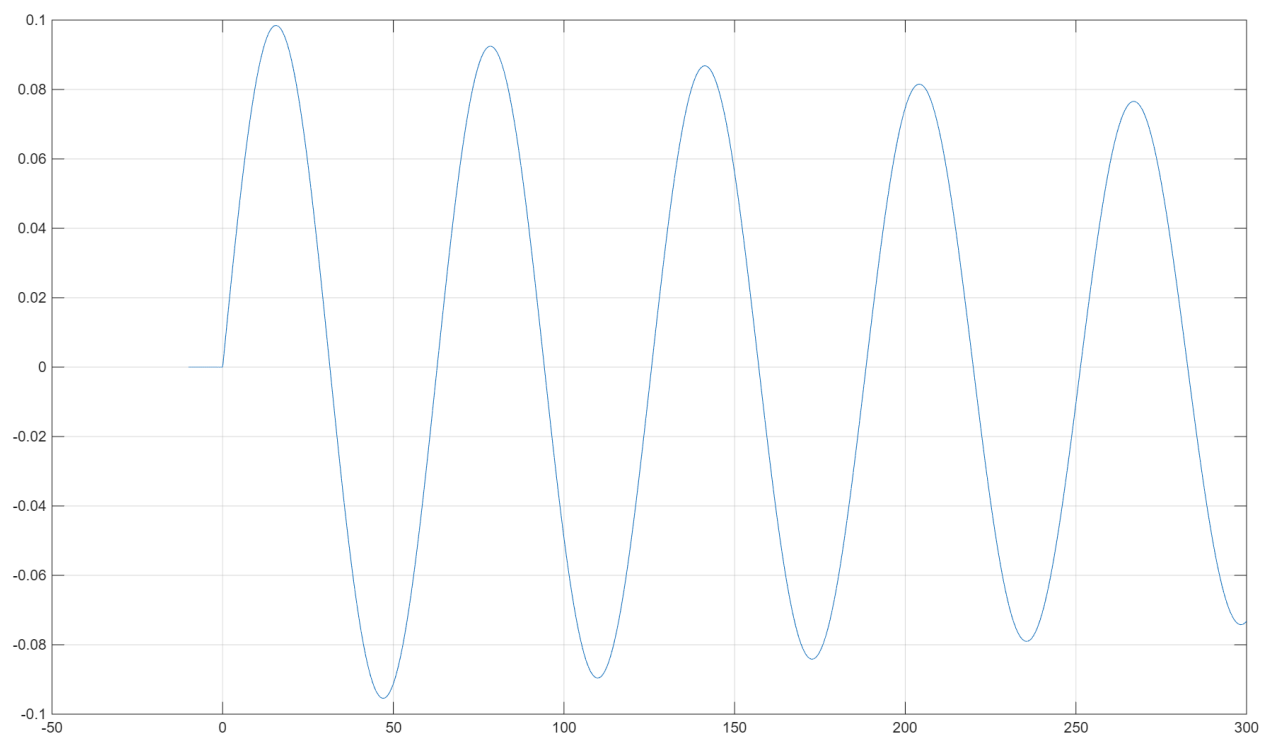
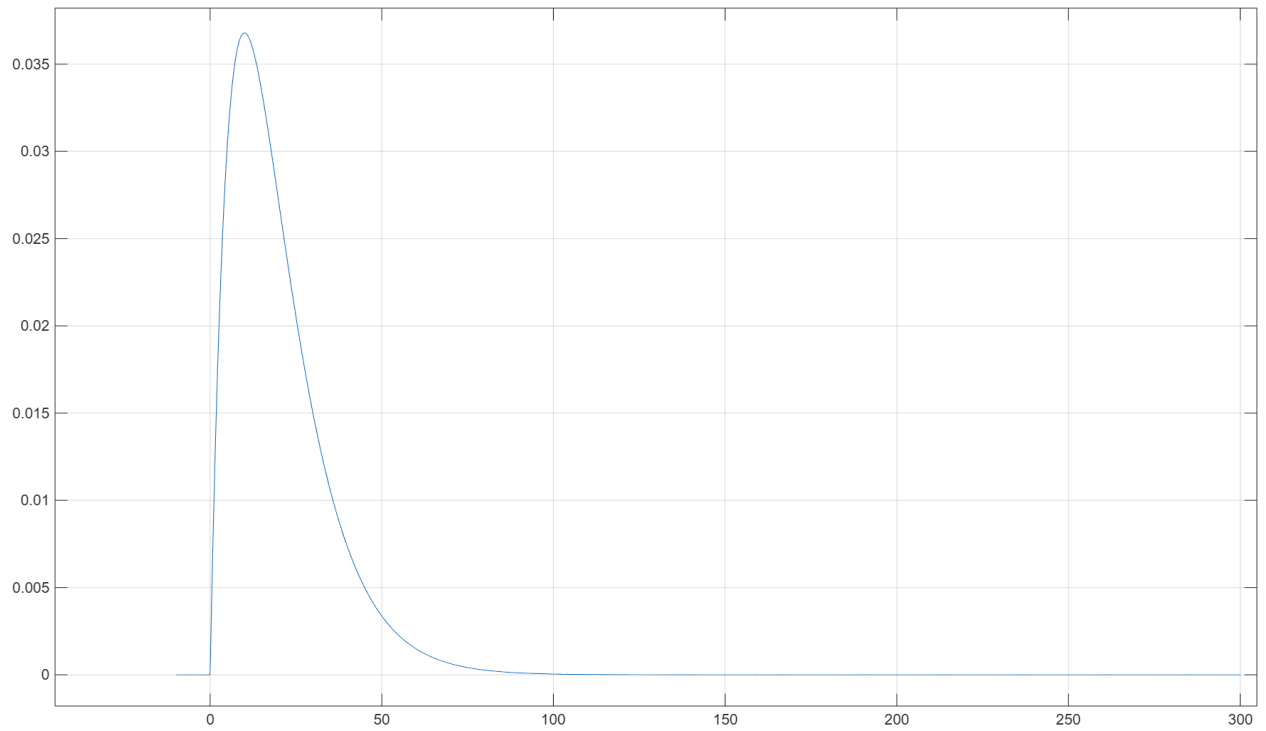


c. Code:

%q3

```
t = -0.1:0.0001:0.1; % Time vector with appropriate resolution
w = sin(400 * pi * t) .* (t >= 0); % Single-tone signal w(t)
x = (cos(100 * pi * t) + sin(400 * pi * t) - cos(800 * pi * t)) .* (t >= 0); %
Received signal x(t)
h = 400 * exp(-200 * t) .* cos(400 * pi * t) .* (t >= 0); % Impulse response
h(t)
y1 = conv(h, x) * 0.0001;
y = y1(1: length(t));
plot(t, x, 'b', 'DisplayName', 'x(t) - received (corrupted)'); hold on;
plot(t, w, 'g', 'DisplayName', 'w(t) - original tone');
plot(t, y, 'r', 'DisplayName', 'y(t) - filter output');
grid on;
xlabel('t (s)');
ylabel('Amplitude');
legend('Location', 'best');
xlim([-0.1 0.1]);
```

4. Let's assume the mass of the damper M is 100 kg, the spring constant k is 1 kgs⁻², and the friction coefficient b is 20 kgs⁻¹. Using Matlab built-in differential equation solver `dsolve()` to derive the impulse response $h_1(t)$ for this Industrial Shock Absorber and the impulse response $h_1(t)$ from a range $-10s \leq t \leq 300s$. Overtime the oil inside the shock absorber degrades and the friction coefficient b becomes 0.2 kgs⁻¹. Derive the new impulse response $h_2(t)$ for this Industrial Shock Absorber and plot $h_2(t)$ from a range $-10s \leq t \leq 300s$
- a. Summary/Discussion: Two mass spring systems were compared using `dsolve`. With $B=20$, the system is overdamped with a response that rises and immediately falls to zero with no oscillation. With $B=0.2$, the system is underdamped with a response that oscillates rapidly and decays very slowly.
 - b. Results:



c. Code:

```
%q4 p1
M= 100;
B = 20;
```

```

K = 1;
syms y(t);
eqn = M*diff(y,t,2) + B*diff(y,t) + K*y == 0;
Dy = diff(y, t);
cond1 = y(0) == 0;
cond2 = Dy(0) == 1/M;
ysol = dsolve(eqn, cond1, cond2);
hfunc = matlabFunction(ysol);
tvals = linspace(-10, 300, 2000);
hvals = zeros(size(tvals));
idx = tvals >= 0;
hvals(idx) = hfunc(tvals(idx));
plot(tvals, hvals)
grid on
hold on
%q4 p2
M= 100;
B = 0.2;
K = 1;
syms y(t);
eqn = M*diff(y,t,2) + B*diff(y,t) + K*y == 0;
Dy = diff(y, t);
cond1 = y(0) == 0;
cond2 = Dy(0) == 1/M;
ysol = dsolve(eqn, cond1, cond2);
hfunc = matlabFunction(ysol);
tvals = linspace(-10, 300, 2000);
hvals = zeros(size(tvals));
idx = tvals >= 0;
figure
hvals(idx) = hfunc(tvals(idx));
plot(tvals, hvals)
grid on
hold on

```